Regular expressions (regex)

Thursday, June 18, 2026

Today: Thursday June 18

Before class

- **Perusall Reading** (Canvas): *Data Computing* Ch. 17 §17.1–17.2 — Regular expressions

Plan for today

- **Project oral check-in #1**; work on the activity if your group doesn't go first.
- What a **regex** and its roles (**detect, replace, extract**)
- literals, `.`, character classes `[ ]`, alternation `|`
- **Quantifiers** `? * + {n}`, **anchors** `^ $`, and **escaping**
- **Capturing** with `( )` and pulling pieces out
- In-class activity

Helpful links

- Syllabus · AI policy · Schedule · *DC* Ch. 17 · stringr cheatsheet · regex101.com

What is a regex?

Patterns in strings

A **regular expression** (regex) is a compact way to describe a **pattern** in text. Lots of everyday strings follow patterns:

Kind	Examples
Phone numbers	`(651) 696-6000` · `+1 651.696.6000`

Kind	Examples
Times	`10:30 AM · 13:15.54`
Durations	`1:59:40` (Kipchoge's sub-2-hour marathon)
US postal codes	`16801 · 55105-1362`
Stray whitespace	`"  henry  "`

A regex lets you say "match anything that has this particular shape" without having to list out every possible case (which is often impossible).

Three jobs regex does

Across all of data cleaning, regex does three things:

Job	base R	stringr (tidyverse)	Often with
Detect a pattern	`grepl()`	`str_detect()`	`filter()`
Replace a pattern	`gsub()`	`str_replace_all()`	`mutate()`
Extract a match	`regmatches()`	`str_extract()`	`mutate()`

The textbook uses base R (`grepl`, `gsub`); the analogous versions from the stringr package do the same things but have tidier names.

grepl(): does the pattern appear?

`grepl(pattern, x)` returns `TRUE`/`FALSE` for each string. The **pattern is the quoted string**. Here: names containing "shine" (`ignore.case = TRUE` ignores capitalization):

```
NameList |>
  filter(grepl("shine", name, ignore.case = TRUE)) |>
  head(5)
```

```
# A tibble: 5 x 3
  name     sex   total
  <chr>    <chr> <int>
1 Sunshine F      4959
2 Shineka  F       150
3 Shine    F        95
4 Shine    M        44
5 Sunshine M        37
```

`str_detect(name, regex("shine", ignore_case = TRUE))` does the same thing.

Regex building blocks

Literals & special characters

The simplest patterns are **literal**: `"able"` matches any string containing "able" ("cap**able**", "t**able**d") but is **case-sensitive** by default ("Able" won't match unless `ignore.case = TRUE`).

These characters are **special** in that they mean something other than themselves:

```
.   ^   $   [   ]   {   }   |   (   )   +   *   ?
```

For example `.` means "any single character", so `".ble"` matches "able", "sol**uble**", "fung**ible**", "B**ible**".

Literals & special characters

Character	Meaning	Example
`.`	**Any single character** (except a newline)	`".ble"` matches "**able**", "sol**uble**", or "B**ible**".
`^`	**Start of the string**	`"^a"` matches "**a**pple" but not "b**a**nana".
`$`	**End of the string**	`"e$"` matches "appl**e**" but not "**e**lephant".
`[ ]`	**Character class** (matches *any one* character inside)	`"[ab]c"` matches "**ac**" or "**bc**".
`|`	**OR** (alternation)	`"cat|dog"` matches "**cat**" or "**dog**".
`( )`	**Grouping** (treats multiple characters as one unit)	`"(ab)+"` matches "**abab**".
`*`	**Zero or more** of the preceding item	`"a*b"` matches "**b**", "**ab**", or "**aab**".
`+`	**One or more** of the preceding item	`"a+b"` matches "**ab**" or "**aab**" (but not "b").
`?`	**Zero or one** of the preceding item (makes it optional)	`"colou?r"` matches "**color**" or "**colour**".
`{ }`	**Quantifier** (specific number or range of repetitions)	`"a{2,3}"` matches "**aa**" or "**aaa**".

> **Note:** If you ever need to match one of these special characters literally (like an actual period or question mark), you have to escape it by putting a backslash in front of it (e.g., `\.` or `\?`). In R, because of how strings work, you usually need *two* backslashes (e.g., `\\.`).

Character classes `[ ]`

Square brackets mean "**any one** of these characters":

- `[aeiou]` → any vowel
- `[^aeiou]` → **not** a vowel (the `^` *inside* brackets means "except") → a consonant
- `[0-9]` → any digit; `[a-z]` → any lowercase letter

```
NameList |>
  filter(grepl("mn", name, ignore.case = TRUE)) |>   # literal "mn"
  head(3)
```

```
# A tibble: 3 x 3
  name   sex    total
  <chr>  <chr>  <int>
1 Autumn F     104408
2 Sumner M       2287
3 Amna   F       1099
```

> **Tip**
>
> A vertical bar `|` means **"or"**: `"rain|snow|sleet|hail"` matches any of those four words.

Putting characters in a row

Two patterns side by side means "this **then** that". `^[^aeiou].[^aeiou].[^aeiou]` reads as:

- `^` start of string, then
- `[^aeiou]` a consonant, `.` anything, `[^aeiou]` a consonant, `.` anything, `[^aeiou]` a consonant

i.e. the 1st, 3rd, and 5th letters are consonants:

```
NameList |>
  filter(grepl("^[^aeiou].[^aeiou].[^aeiou]", name, ignore.case = TRUE)) |>
  head(3)
```

```
# A tibble: 3 x 3
  name   sex     total
  <chr>  <chr>   <int>
1 James  M     5091189
2 Robert M     4789776
3 David  M     3565229
```

Quantifiers

How many times?

A quantifier follows a pattern and says **how many times** it repeats:

Quantifier	Means
`?`	zero or **one** time
`*`	zero or **more** times
`+`	**one** or more times
`{n}`	exactly `n` times
`{n,m}`	between `n` and `m` times
`{n,}`	`n` or more times

> **Warning**
>
> The *Data Computing* reading swaps the definitions of `+` and `*`. **In R (and every real regex engine):** `*` = zero-or-more, `+` = one-or-more.

Quantifiers in action

```
x <- c("Mr.", "Ms", "Mrs.", "Miss", "Man", "Miocene")
x[grepl("M[aeiou]+", x)]   # M then ONE or more vowels
```

```
[1] "Miss"    "Man"     "Miocene"
```

```
x[grepl("M[aeiou]*", x)]   # M then ZERO or more vowels (matches Mr.)
```

```
[1] "Mr."     "Ms"      "Mrs."    "Miss"    "Man"     "Miocene"
```

Three or more vowels in a row, `[aeiou]{3,}`:

```
NameList |>
  filter(grepl("[aeiou]{3,}", name, ignore.case = TRUE)) |>
  head(3)
```

```
# A tibble: 3 x 3
  name   sex    total
  <chr>  <chr>  <int>
1 Louis  M     389910
2 Louise F     331551
3 Isaiah M     177412
```

Anchors & escaping

Start `^` and end `$`

Outside of square brackets, `^` and `$` are **anchors**:

- `^` = the **start** of the string
- `$` = the **end** of the string

Names that **end** in a vowel, `[aeiou]$`:

```
NameList |>
  filter(grepl("[aeiou]$", name)) |>
  group_by(sex) |>
  summarise(total = sum(total), .groups = "drop")
```

```
# A tibble: 2 x 2
  sex      total
  <chr>    <int>
1 F     96702371
2 M     21054791
```

Girls' names end in a vowel almost five times as often as boys' names.

Escaping special characters

To match a special character **literally**, **escape** it with `\\`. So `\\.` means a real period, and `\\$` a real dollar sign (not the end-of-string anchor).

```
currency <- c("$100.95", "45¢")
gsub("^\\$|¢$", "", currency)   # strip a leading $ or trailing ¢
```

```
[1] "100.95" "45"
```

Here `^\\$` is a literal `$` at the start; `¢$` is a literal `¢` at the end. Without the `\\`, that first `$` would mean "end of string".

Capturing & extracting

Parentheses capture

Wrap part of a pattern in `( )` to **capture** it; in other words, pull that piece out. `str_extract()` returns the matched text; with capture groups, `gsub()` can refer back to group 1 as `\\1`.

Names with **four vowels in a row**, extracting those vowels:

```
BabyNames |>
  group_by(name) |>
  summarise(total = sum(count), .groups = "drop") |>
  mutate(match = str_extract(name, "([aeiouAEIOU]{4})")) |>
  filter(!is.na(match)) |>
  arrange(desc(total)) |>
  head(4)
```

```
# A tibble: 4 x 3
  name    total match
  <chr>   <int> <chr>
1 Louie   29293 ouie
2 Sequoia  3275 uoia
3 Gioia     480 ioia
4 Keiaira    81 eiai
```

Backreferences with \\1

The regex `".*([aeiou])$"` = "anything, then a captured final vowel". In `gsub()`, `replacement = "\\1"` keeps **only** the captured group, so the whole name collapses to its last vowel:

```
re <- ".*([aeiou])$"
NameList |>
  filter(grepl(re, name)) |>
  mutate(vowel = gsub(re, "\\1", name)) |>
  group_by(sex, vowel) |>
  summarise(total = sum(total), .groups = "drop") |>
  pivot_wider(names_from = sex, values_from = total)
```

```
# A tibble: 5 x 3
  vowel        F        M
  <chr>    <int>    <int>
1 a     56088501  1844041
2 e     36432218 14341114
3 i      3693024   753311
4 o       403120  4041190
5 u        85508    75135
```

Names ending in "a", "e", "i" skew female; "o" skews male.

Cleaning with regex

Stripping junk out of numbers

Scraped numbers arrive with `$`, `,`, `%`, units. A character class lists everything to delete at once:

```
debt <- tibble(
  country = c("World", "United States*", "Japan"),
  debt    = c("$56,308 billion", "$17,607 billion", "$9,872 billion"),
  percGDP = c("64%", "73.60%", "214.30%")
)
debt |>
  mutate(
    debt    = gsub("[$,%]|billion", "", debt),
    percGDP = gsub("[,%]", "", percGDP)
  )
```

```
# A tibble: 3 x 3
  country        debt     percGDP
  <chr>          <chr>    <chr>
1 World          "56308 " 64
2 United States* "17607 " 73.60
3 Japan          "9872 "  214.30
```

`[$,%]` deletes any `$`, `,`, or `%`; `|billion` also deletes the word "billion".

Trimming whitespace

`^ +| +$` matches one-or-more spaces at the **start** *or* the **end**:

```
s <- "   My name is Julia     "
gsub("^ +| +$", "", s)
```

```
[1] "My name is Julia"
```

> **Tip**
>
> stringr has friendly shortcuts for these: `str_remove_all()`, `str_trim()` (ends), and `str_squish()` (ends + collapse internal runs). Same regex ideas underneath.

Activity

In-class activity

Open `day23-activity.R` in RStudio and work through the parts in order. (Work on the activity when it isn't your group's turn for **oral check-in #1**.)

- **Part 1**: **detect** patterns with `grepl()` / `str_detect()` + `filter()` on `BabyNames`.
- **Part 2**: build patterns from **character classes, quantifiers, and anchors** (including a "Caitlin"-style name and a phone-number regex).
- **Part 3 (challenge)**: **extract** with capture groups, and **clean** messy strings with `gsub()`.

Activity file

Wrap-up & looking ahead

What we covered

- A regex describes a **pattern**; it does three jobs: **detect / replace / extract**
- `.` any char · `[aeiou]` class · `[^aeiou]` negation · `|` or
- Quantifiers `?` `*` `+` `{n}` `{n,m}` `{n,}` (R: `+` = one-or-more, `*` = zero-or-more)
- Anchors `^` start, `$` end; **escape** specials with `\\`
- Capture with `( )`; pull pieces with `str_extract()` or `\\1` in `gsub()`
- Clean numbers/whitespace with `gsub()` (or `str_remove_all`/`str_squish`)

Upcoming

- **Fri Jun 19: No class — Juneteenth**
- **Quiz 5** rescheduled to **Mon Jun 22**
- Keep building your **project**; oral check-in #1 today

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *DC* Ch. 17 · regex101.com